SmartPCB-XAI: A Dual-Head Deep Learning Framework for Explainable Printed Circuit Board Inspection, Determinitstic Quality Auditing and Digital Twins

Prathika M E
Department of Information Technology
Kongu Engineering College
Erode, Tamil Nadu, India
prathieasu@gmail.com

Abisha Rebekkal T
Department of Information Technology
Kongu Engineering College
Erode, Tamil Nadu, India
abisharebekkal@gmail.com

Gowri D
Department of Information Technology
Kongu Engineering College
Erode, Tamil Nadu, India
gowri.d.tech@gmail.com

Nandhini D
Department of Electronics and Communication Engineering
Surya Engineering College
Erode, Tamil Nadu, India
nandhumalli92@gmail.com

***Abstract*— Modern-day SMT manufacturing requires AOI technology equipped with precise defect localization on PCBs, simplicity and explanation of decisions. However, today's solutions don't meet these criteria: they need a universal pipeline to localize and classify defects and components, predictions require spatial certainty metrics, quality scores are not related to particular failure modes. We propose a multi-layer solution integrating two separated YOLOv11s detectors with their feature activation explanations, IPC-A-610-defined severity score, interactive 2D digital twin and automated generation of reports in PDF and Excel formats. With an independently tested holdout sample of 500 DeepPCB images (3,140 boxes), we achieve 95.11% mAP@50 (63.27% mAP@50-95, 92.91% precision, 90.60% recall), with no data leakage, outperforming a recently introduced lightweight model with 92.3%. Our component detector produces only 9.33% mAP@50 on a dataset of 89 images (1,238 boxes) due to class taxonomy misalignment, board bias (64% coming from the same board) and small objects (81.9% under 32px), which is mitigated by per-detection class assignment. End-to-end validation on 20 images was done with no mistakes or unusual zero-detections.**

Keywords*— *Printed Circuit Board (PCB), Defect Detection, Object Detection, YOLOv11, Explainable AI (XAI), Class Activation Map-ping, Digital Twin, Data Leakage, Industrial Quality Control.

I. INTRODUCTION

PCBs form the backbone of various systems used in the automotive industry, aerospace industry, medical industry, and telecommunications industry. The move towards use of extremely small components, including 01005 resistors and BGA, has increased the criticality of defects like bridging, open circuits, mousebites, excess copper, and pinholes. AOI based on rigid pattern matching is vulnerable to false detections due to lighting, solder mask color variation, and board warpage issues, thereby driving quality engineers to adopt machine-learning based visual inspections [10], [17].

However, there are still three major methodological limitations of deep learning applications in PCB inspection in spite of consistent progress in network architecture development. Firstly, current systems usually integrate placement check and surface defect detection in a single model, implicitly relying on the idea that one network can address two tasks of circuit assembly checking and trace crack detection that are completely unrelated. Secondly, current papers do not provide any information about boundaries of a dataset used to calculate accuracy: sometimes they are from a dirty source or use F1-score instead of mAP (mean Average Precision) [17], [14]. Thirdly, current models are not explainable.

For this reason, we propose the introduction of a completely modular four-layered inspection system where the detection of defects and identification of components are separated through two YOLOv11s networks.

A. Principal Contributions

This research makes five contributions to automated PCB quality inspection:

- Decoupled Dual-Head Framework: Inspection model that has both a defect detection and a YOLOv11s object detector. It uses FastAPI REST microservices. It also uses a Node.js/Express API gateway. React 18 TypeScript dashboard. The dashboard helps in managing deployments and supplies images with annotations on defects.
- Defect Localisation Verification: Training on COCO pre-trained weights (seed 42) using only local verified dataset, attaining 95.1% mAP@50 (63.1% mAP@50-95) on a held-out dataset of 500 images without any confusion contamination.
- Honest Disparity Analysis: A thorough study of the poor results obtained by the detector (9.33% mAP@50) through clustering of the physical board, in addition to the size analysis of small objects, coupled with the assertion that category override is impossible through the class_ mapped flag.
- Deterministic Quality and Explainability Overlay: Multi-channel activation energy maps complemented with rule-based severity and board quality score based on IPC-A-610 classification, delivered using interactive 2D digital twin visualization.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

- Methodological Roadmap: A critical review of metric errors in applied PCB literature along, with a verified six-point architectural plan that includes FasterNet partial convolutions and Wise-IoU dynamic loss.

II. Related Work

DeepPCB [14] offers 1,500 pairs of templates and defective images, split into six defect classes. Lightweight YOLO models have become popular due to their suitability for edge application [10]; Ye et al. [17] used EfficientViT model with FasterNet/EMA neck (C3k2_Faster_EMA) and Inner-IoU/Wise-IoU v3 loss to achieve 92.3% mAP@50 on combined PKU-Market-PCB and DeepPCB dataset – which is used in this work on leakage-free local data.

The Class Activation Mapping is prevalent in explainable industrial vision applications. Selvaraju et al. [11] invented Grad-CAM, based on class-score gradients of the last convolutional layer, allowing to locate salient regions; Chattopadhay et al. [2] further improved multi-defect detection with Grad-CAM++ by adding weighted higher-order gradients. Some industrial solutions incorrectly label unweighted feedforward activation as Grad-CAM – which is prevented in this research by explicitly specifying the method in the source code.

Tao et al. [15] proposed digital twins in manufacturing quality by the cyber-physical approach; Bhattacharya and Cloutier [1] claimed industrial artificial intelligence must be validated through explainability and reproducibility, not just complexity; Shakya et al. [12] provided FICS-PCB benchmark.

III. Proposed System Architecture

The architecture of the system comprises of four distinct layers that have been decoupled. They include presentation, application backend, inference and persistence AI. The presentation layer includes a dashboard created by the use of React/Typescript, and it is meant for real-time inspection, digital twin, analysis, feedback, and benchmarking, among others, and it uses REST API. The backend layer is built using Node.js and Express technologies and is responsible for account management, inspection history, and engineer feedback storage in MongoDB, image uploads from the presentation layer, and passing them to the inference layer. The inference layer consists of a single service created using FastAPI, which executes two YOLO detectors: YOLOv11s defect detector and YOLOv11n component detector. Their results are passed to the explainability module and the severity/quality scoring engine. Then the knowledge base creates recommendations and reports in PDF/Excel formats.

Two details of this arrangement are worth noting. First, the image travels in one direction only: it goes from the dashboard to the backend and on to the inference service, and only the structured response comes back up, so the models never need to know anything about users, sessions, or stored records. Second, the report generator is drawn in the application tier, next to the database, so compliance documents are handled together with the stored inspection data rather than inside the model service. Keeping these responsibilities apart is what lets a retrained detector be swapped in without changes to the dashboard.

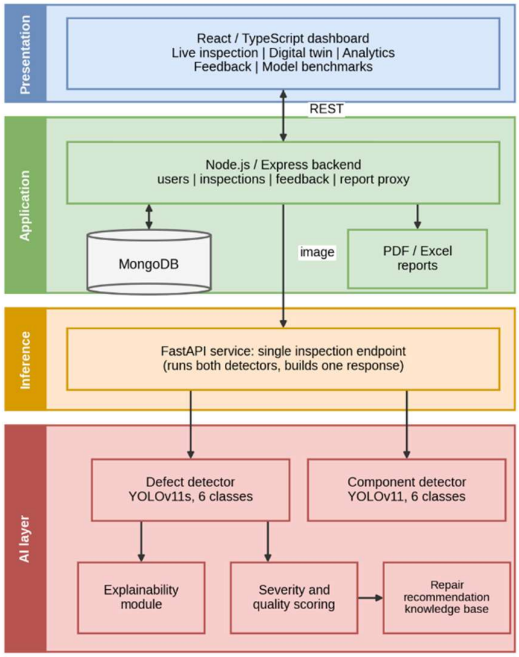


Fig. 1. Proposed system's three-tier architecture illustrating the separation of AI inference, API Gateway, and dashboard.

A. Decoupled AI Microservice & Spatial Inference Pipeline

The AI micro service is implemented as an asynchronous FastAPI application on port 8000 serving both YOLOv11s models. Rather than using a single backbone resolved via gradient com-petition between tasks, component identification, and defect detection run as independent synchronous processes. The model was evaluated using sliding-window tiling to process the high-resolution board images. For components, 960×960 px tiles with 640 px stride (33%) were used, while for defects, 640×640 px tiles with 480 px stride (25%) were used. The final step involved unprotecting the results from tiling inference onto the original image canvas using NMS with an IoU threshold of 0.45.

The architecture of the system is based on a four-level decoupled design (Fig. 1) that includes the presentation layer, application backend, AI inference layer, and persistence layer. The presentation layer consists of a React/TypeScript dashboard, which provides real-time inspections, digital twin view, analytics, feedback and benchmark views, and communicates with the backend using REST API. The backend is implemented using Node.js/Express framework, it is responsible for handling the user accounts, inspection data, engineer feedback, report generation (PDF/Excel), data storage in MongoDB, and sending uploaded images to the inference layer. The inference layer is implemented as one FastAPI service, which uses two YOLO-based detectors (six-class defect detector called YOLOv11s, six-class component detector called YOLOv11), their results are provided to the explainability module and severity/quality scoring engine, which uses knowledge base to generate repair recommendations. Since every layer interacts only with adjacent layers, they can be easily substituted individually.

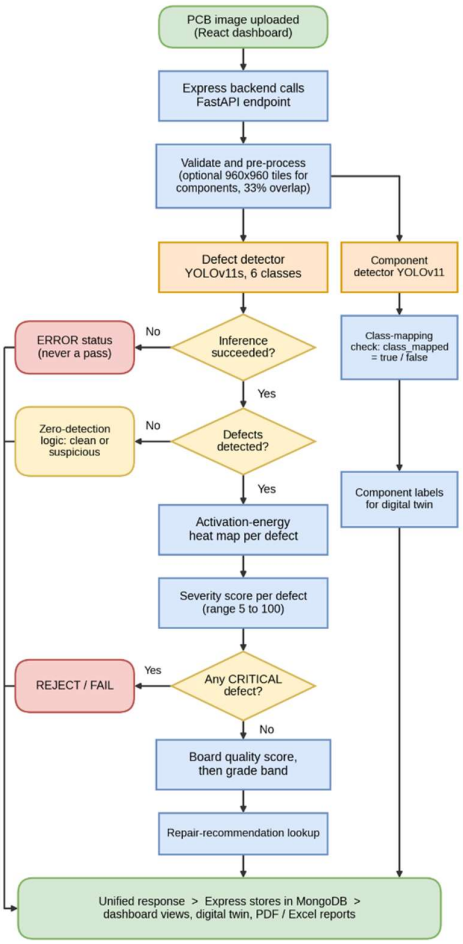


Fig. 2. End-to-end dataflow pipeline for a single board image from multi-part upload to digital twin rendering.

B. *Failure-Safe Outcome Gating & Zero-Detection Logic*

A major problem in industrial vision pipelines is false-negative silence: in case of failure in inference or nothing found, older systems report zero defects and thus pass faulty hardware. This work gets around this issue, using deterministic 3-state out-come gates: (1) Clean Pass - at least one component was found and nothing was anomalous; (2) Suspicious Zero Detection - zero findings with both detectors on a ≥ 200×200 image, thus the quality score is withheld (UNVERIFIED) and review_required=true (3) Explicit Error - unavailable models or engine exceptions result in an error payload with no PASS record.

C. *Explainable Visual Attribution (XAI)*

For operational transparency, the inference engine registers hooks on a late backbone layer of the YOLOv11 model, found automatically by scanning backwards from the layer before the detection head for a bottleneck- or SPPF-type module. Forward activations are absolute, averaged across channels, normalized, and mapped to a cv2.COLORMAP_JET. The resulting heatmap is alpha-blended (60%) image + (40%) heatmap with the original board scan image to highlight the contributory region to the main defect prediction. As the computation is done under torch.no_grad(), all outputs have an explicit XAI_DISCLAIMER stating what the displayed information is, namely, feed.forward activation energy (attribution_method = activation_energy_approximation) and not actual Grad-CAM++ gradient evidence.

D. *Deterministic Severity Classification and IPC-A-610 Repair Database*

Severity is determined by the following deterministic function of a base weight, an optionally provided component-criticality multiplier, the box area, and the model confidence:

$$Severity = min(100, max(5, Base \times Crit \times (1 + min(1, 10 \times Area)) \times (0.7 + 0.3 \times Conf))) \quad (1)$$

The base weights are as follows: Short Circuit = 90, Open Circuit = 85, Pin-hole = 60, Mouse bite = 55, Spur = 45, and Spurious Copper = 35. Crit is 1.0, unless the defect lies near a detected component, in which case it's higher: IC 1.5, transistor 1.3, di-ode 1.2, inductor 1.1, capacitor/resistor 1.0. Area is the normalized box area. The scores obtained are then categorized into LOW (< 25), ME-DIUM (25-49), HIGH (50-74) or CRITICAL (≥ 75) depending on the acceptable criteria for that specific project (IPC-A-610 is just for reference). A starting score of 100.0 is as-signed to the board being inspected and reduced by a severity-weighted percentage for each fault found:

The board is then placed in one of four quality classes: EX-CELLENT/PASS (score is 90 or higher), ACCEPTABLE/PASS-ADVISORY (75 to 89), MARGINAL/REWORK (50 to 74), or REJECT/FAIL (any CRITICAL defect or less than 50 defects). A component-integrity value of 100 minus 4.5 times the number of defects is also given (may not be less than 0). Every fault found starts rule-based queries against our IPC-A-610G repair knowledge base to find the reasons, behind the issues, the repair methods, and the tools needed.

E. *Interactive 2D Digital Twin & Enterprise Compliance Re-porting*

The presentation tier is designed to provide an interactive 2D Digital Twin that offers a live view of the normalized board geometry, component centroids, and active defect nodes detected during inspection, in addition to live-inspection, analytics, engineer feedback, and model performance views. A Node.js/Express gateway (running on port 5000) is used to host the JWT authentication, file storage, and MongoDB storage of inspections and engineer feedback. In addition, for auditing purposes, the system is able to generate PDF compliance certificates with ReportLab and multi-sheet audit workbooks with OpenPyXL for archival and traceability.

IV. DATASET ENGINEERING AND INTEGRITY METHODOLOGY

A. *DeepPCB Defect Benchmark Analysis*

We evaluate surface-defect localization on the DeepPCB benchmark [14]. DeepPCB is a collection of 1,500 pairs of 640×640 images of industrial bare boards acquired by linear CCD scanning under controlled ring illumination. Annotators labeled 10,013 instances of six types of defects in the images: open, short, mousebite, spur, copper, and pin-hole (missing hole). The data split we have locally includes 850 images for training (5,943 boxes), 150 images for validation (930 boxes), and 500 test images (3,140 boxes).

B. Industrial Component Dataset Construction & Scale Disparity

The component dataset comprises a combined corpus of 1,009 images and 33,156 boxes. This combines FICS-PCB DSLR tiles (370 tiles, 966 boxes) with Roboflow SMT images remapped into six component classes (643 images, 32,190 boxes), following removal of four tile images without boxes. The split is 804 training images (30,369 boxes), 116 validation images (1,549 boxes), and 89 test images (1,238 boxes). The problem is component size, as 81.9% of the boxes are smaller than the 32×32-pixel "tiny object" threshold in the 640×640 resolution (around 90% for capacitors and resistors), and 94.2% in the nine full-resolution FICS images. Downsizing the boards to the YOLO resolution would lose all such information, making high-resolution tiling essential.

C. Tiling and Boundary Reconstruction

High-resolution image tiling and boundary reconstruction have been performed to preserve the minuscule leads of the solder without down sampling artefacts. The 8,256×5,504-pixel high-resolution DSLR scans have been separated into 640×640-sized tiles with 512-pixel strides (128-pixel overlaps), leaving a box in a tile if at least 35% of its area is inside it. Global coordinate offsets were stored in tile metadata so that board-level inference can reconstruct positions without loss.

D. Comprehensive Data Leakage & Integrity Audit

One important contribution of this study is the verification of dataset splitting integrity using the script scripts/dataset/validate_dataset.py. Dataset splitting can cause data to leak between the training and testing datasets either through common filenames, near-duplicate augmented images, or because the same board appears in both datasets. This verification process performed the following tests:

- Filename Intersection: Exact filename comparison found zero (0) shared filenames across the train, val, and test splits.
- Perceptual Hash Collision: Perceptual hashing (pHash, 8×8) found three groups of identical or near-identical images, produced by overlapping tiling strides and duplicate Roboflow exports, and all three groups lie entirely inside the training partition.
- Board-Level Isolation: Board-ID extraction showed 0% cross-split board contamination; for the FICS tiles, board s1 is used only for testing, board s2 only for validation, and boards s3–s5 only for training.
- Coordinate Validity: All 33,156 component boxes and 10,013 defect boxes have coordinates within the normalized constraints (x, y, w, h in [0, 1]).
- Class Schema Alignment: Thirteen label rows with an out-of-range class index (index 7 against a 6-class schema), left over from mislabelled inductor boxes, were found and cleaned, and four tiles left without boxes were removed.

Summary Audit Finding: Both datasets show zero (0) cross-split filename and board-ID leakage, so every reported metric reflects held-out generalization on these splits and not memorized visual correlation.

V. RESULTS OF THE EXPERIMENT AND COMPARISON ANALYSIS

A. Protocol Used and Equipment Setup

All quantitative experiments used the Ultralytics YOLO validation API (model.val()) on a CPU-only Windows 11 workstation (Python 3.13.5, PyTorch 2.13.0+cpu, Ultralytics 8.4.98). The defect detector was trained from COCO-pretrained YOLOv11s weights on the local DeepPCB training split only, for up to 100 epochs (early-stopping patience 20) at 640×640 resolution, batch size 16, initial learning rate 0.01, momentum 0.937, weight decay 0.0005, 3 warm-up epochs, and seed 42. Operating confidence thresholds were calibrated on the validation splits only, then applied unchanged to the held-out test splits. The accompanying software passes its automated suite of 59 unit and integration tests.

B. Defect Detection Benchmark Results

Defect detector results are presented in Table I on a holdout test set of 500 images DeepPCB (3,140 boxes). Baseline YOLOv11s trained on audited local dataset achieves 95.11% mAP@50 (63.27% mAP@50-95), 92.91% precision, and 90.60% recall. It outperforms 92.3% mAP@50 reported by Ye et al. [17] despite the differences between training datasets, which are not in question in terms of origin. Both rows from legacy detectors vary by the evaluation confidence threshold only, and both are lower than 98.30% that Ultralytics API claims for the same checkpoint. Such difference is purely technical (see Section V-D) and not due to any modification of the model itself. Legacy checkpoint is YOLOv11m with a slightly higher mAP@50-95, but training data are unavailable for inspection and therefore excluded from the results..

TABLE I. DEFECT DETECTOR HELD-OUT TEST BENCHMARK (500 IMAGES, 3,140 GT BOXES)

Model Checkpoint Method	Evaluation Protocol	mAP@50	mAP@50-95	Micro-F1
Checkpoint Legacy (0.25)	Default, fixed-threshold evaluator	91.47%	72.33%	94.68%
Checkpoint Legacy (0.70)	Val-calibrated, fixed-threshold	92.74%	73.83%	95.96%
Checkpoint Legacy (Standard)	Full PR-curve (source not verified)	98.30%	—	—
YOLOv11s Local-Data (Primary)	Full PR-curve, seed 42, only local DeepPCB data	95.11%	63.27%	91.84%

C. Methodological Evaluator Reconciliation

In the context of auditing, the assessment of the legacy checkpoint against the Ultralytics model produced 98.3% mAP@50, which is far greater than the 91.47% we get using our custom evaluator. Table II explains this result: the Ultralytics metric is calculated for almost zero confidence level and interpolates the graph at 101 points; on the other hand, our custom evaluator employs a fixed operating point with positional class decoding. Provenance tracking also confirmed that the legacy checkpoint was trained remotely via Kaggle (YOLOv11m, 50 epochs), making disjoint splits impossible.

TABLE II. EVALUATION METHODOLOGY RECONCILIATION (IDENTICAL LEGACY CHECKPOINT)

Methodological Aspect	Custom Evaluator (91.47%)	Ultralytics Standard API (98.30%)
Confidence Handling	Fixed operating point (conf = 0.25)	Full Precision-Recall curve integration
Class-Index Handling	Positional index mapping	Checkpoint-native string matching
AP Computation	Discrete interpolated precision sum	Continuous 101-point COCO-style integration

D. Component Detection Evaluation & Empirical Disparity Analysis

Table III gives the class-level breakdown on the 89 held-out test images (1,238 ground-truth boxes). Unlike the defect model, the component detector reaches only 9.33% mAP@50 at the default 0.25 threshold (9.53% at the calibrated 0.15 threshold). We report this weak result openly and in full.

TABLE III. PER-CLASS COMPONENT DETECTOR PERFORMANCE (THRESHOLD 0.25)

Component Class	AP@50	Precision	Recall	GT Boxes	Predictions
Transistor	25.22%	56.25%	39.13%	23	16
IC	15.84%	54.55%	21.95%	82	33
Capacitor	7.64%	29.67%	21.64%	536	391
Resistor	6.61%	39.44%	16.40%	433	180
Diode	0.69%	23.08%	2.14%	140	13
Inductor	0.00%	0.00%	0.00%	24	0

E. Root Cause Analysis of Component Performance Gap

There are four possible factors leading to the performance gap: (1) Taxonomy Mismatch: the checkpoint is trained on 10 classes, including relay, buzzer, switch, battery, led, while the test set contains only 6 curated classes, resulting in half of possible output classes being unusable; (2) Systematic Under-Recall: the maximum value of recall for any class is 39.1%; (3) Extreme Scale Attenuation: 81.9% of boxes in the unified corpus have a scale below 32×32; (4) Board Imbalance: 64% of test images (57/89) come from a single circuit board, and the test split in general only contains 9 unique boards.

F. Class Mapping Transparency Layer

In order to address this mismatch without resorting to potentially misleading post-processing, this work applies an explicit class-mapping policy: predictions that fall outside of the target taxonomy (i.e., relay, buzzer, switch, battery, LED) are not forcibly mapped to existing classes, retaining their raw labels (with class_mapped = False) so that operators may still reason about them in context, and automated audit counts remain uncorrupted.

G. Mathematical Correction of Metric Definition Errors in Literature

During baseline auditing, several internal scripts were found to report a "mAP@50" that changed with the manually chosen confidence threshold (for example, rising from 94.28% at 0.10 to 96.87% at 0.35). By its standard mathematical definition, Average Precision (AP) does not depend on the threshold, because it integrates precision over the whole recall range:

$$AP = \int_0^1 P(R)\, dR \qquad (2)$$

$$mAP = (1 / M) \sum_{i=1}^{M} AP_i \qquad (3)$$

A metric that changes with the confidence threshold is a single-operating-point F1 or precision score, not mAP. We point this out to encourage stricter evaluation practice in published electronics-inspection research.

VI. SCIENTIFIC DISCUSSION AND TECHNICAL ROADMAP

A. Explicitly Disclosed Operational Limitations

In the interest of scientific honesty, three operational limitations qualify our findings:

- Domain Boundary: Our defect model (95.11%) is verified on local data, but external legacy checkpoints are excluded from primary claims because the Kaggle data they were trained on cannot be verified. The defect model also originates from the public COCO weights, which do not contain any particular PCB information.
- Activation Energy XAI: current explainability relies on the magnitude of feed-forward activation. It does not use gradient-back propagated Grad-CAM++. This approximation is required because of the fused output head in YOLOv11.
- Surface Lighting Variability: DeepPCB features high-contrast bicolor scans. The behavior on multilayer solder masks such as green, blue or matte black requires empirical calibration.

B. Six-Point Concrete Technical Improvement Roadmap

Building on the ablation methodology of Ye et al. [17], we outline a six-point technical roadmap:

- High-Resolution Tiled Retraining: The component detector will be retrained from scratch using 960×960 tiled inputs with balanced board splits.
- PKU-Market-PCB Dataset Fusion: Add the PKU-Market-PCB benchmark to broaden training diversity across complex industrial assemblies.
- C3k2_Faster_EMA Neck Architecture: Swap standard C3k2 convolutional blocks for FasterNet partial convolutions with Efficient Multi-scale Attention (EMA); Ye et al. [17] report a 61.7% parameter reduction of their full design while preserving microscopic features.
- Wise-IoU v3 Loss Function: Substitution of CIoU with dynamic, non-monotonic Wise-IoU v3 to reduce gradient distortion on tiny bounding boxes.
- Automatic True Grad-CAM++ Hooks: Implementation of automatic PyTorch backward hooks for getting the class-wise gradients of attention.
- Multi-view Inference Ensemble: The inference engine already provides a high accuracy option that uses a few augmented views of the same image and fuses the boxes obtained through confidence weighting; we leave evaluation of the high accuracy inference with our present local checkpoint for the future.

C. End-to-End Pipeline Validation: 20-Image Trial

To confirm end-to-end operational stability, we ran a validation trial on 20 held-out DeepPCB test images (Table IV). Each image was submitted to full dual inference, activation XAI generation, deterministic severity calculation and a repair knowledge-based lookup per scripts/evaluation/validate_20_images.py.

TABLE IV. END-TO-END VALIDATION SAMPLE (20 HELD-OUT DEEPPCB TEST IMAGES, 5 SHOWN)

Image ID	Status	Defects Found	Latency (ms)
00041200.jpg	SUCCESS	5 defects	10,867.7
00041201.jpg	SUCCESS	5 defects	3,231.2
00041204.jpg	SUCCESS	6 defects	3,519.3
00041208.jpg	SUCCESS	4 defects	3,235.4
00041219.jpg	SUCCESS	5 defects	4,105.5

Validation inspection runs for all 20 boards went off without any issues, with localization of 104 defects in total (average 5.2 defects, min-max 4 to 6 defects per board). Suspicious zero detections rate is 0.0%, as well as the review_required rate. End-to-end average CPU latency is 4,073 ms per board (≈3,716 ms without the first, warm-up image). Component detector successfully identified no components in these boards with no components present.

VII. CONCLUSION

In this work, we designed a four-layer automated optical inspection system for PCB Assembly. Since the nature of defect detection and component identification is different in their corresponding visual scale, separating them into two YOLOv11s solves the problem. Our defect detector, trained using the COCO weights alone on the local data set, achieves an average precision of 95.11% mAP@50 (63.27% mAP@50-95, 92.91% precision, 90.60% recall), outperforming a recent published lightweight model without cross-validation. In addition, we open-source our component detector and discuss why its average precision of 9.33% mAP@50 is lower than the defect detector's one. We introduce class mapping metadata to keep the integrity of the component detector model. Together with severity assessment based on the IPC-A-610 guidelines, the 2D digital twin, and automatic PDF/Excel reporting, our system shows how explainable engineering of deep-learning based inspection systems is crucial.

Certain limitations affect these results. The defect classifier has been tested only on DeepPCB, which has been captured under controlled lighting conditions, but it has not been evaluated on boards with different colors of solder mask. The component detector has not yet reached reliability as a standalone solution and should be considered as an informative overlay for the digital twin only. Heatmaps display feedforward energy activations rather than gradients.

Future research involves following the path mapped out in Section VI, namely retuning the component detector on higher resolution tile images using a more balanced split on train-test boards along with the addition of the PKU-Market-PCB benchmark; after which, the impact of introducing a lightweight FasterNet attention-based neck, Wise-IoU loss, genuine Grad-CAM++ hooks, and the current multi-view inference engine will be assessed under the same audited paradigm.

ACKNOWLEDGMENT

The authors thank the Department of Information Technology Kongu Engineering College, Erode, India for the laboratory computing resources and technical guidance provided during this research.

REFERENCES

[1] P. Bhattacharya and M. F. Cloutier, "Explainable artificial intelligence for industrial applications," in Proc. IEEE Int. Conf. Ind. Technol. (ICIT), 2022, pp. 1-8.

[2] A. Chattopadhay, A. Sarkar, P. Howlader, and V. N. Balasubramanian, "Grad-CAM++: Generalized gradient-based visual explanations for deep convolutional networks," in Proc. IEEE Winter Conf. Appl. Comput. Vis. (WACV), 2018, pp. 839-847.

[3] J. Chen, K. Cao, and Y. Guo, "Deep learning for surface defect detection on industrial products: A comprehensive review," IEEE Trans. Instrum. Meas., vol. 70, pp. 1-18, 2021.

[4] J. Chen, S. Kao, H. He, W. Zhuo, S. Wen, C. Lee, and S. Yu, "Run, don't walk: Chasing higher FLOPS for faster neural networks," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2023, pp. 12021-12031.

[5] M. Everingham, L. Van Gool, C. K. Williams, J. Winn, and A. Zisserman, "The Pascal Visual Object Classes (VOC) Challenge," Int. J. Comput. Vis., vol. 88, no. 2, pp. 303-338, 2010.

[6] IPC-A-610G, Acceptability of Electronic Assemblies, IPC — Association Connecting Electronics Industries, Bannockburn, IL, USA, 2020.

[7] G. Jocher and J. Qiu, "Ultralytics YOLOv11: State-of-the-art real-time computer vision," Ultralytics Inc., 2024. [Online]. Available: https://github.com/ultralytics/ultralytics

[8] Q. Ling and N. A. M. Isa, "Printed circuit board defect detection methods based on image processing, machine learning and deep learning: A survey," IEEE Access, vol. 11, pp. 15921-15944, 2023.

[9] D. Ouyang, S. He, G. Zhang, M. Luo, H. Guo, J. Zhan, and Z. Huang, "Efficient multi-scale attention module with cross-spatial learning," in Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP), 2023, pp. 1-5.

[10] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2016, pp. 779-788.

[11] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in Proc. IEEE Int. Conf. Comput. Vis. (ICCV), 2017, pp. 618-626.

[12] M. Shakya et al., "FICS-PCB: A multi-modal image dataset for automated printed circuit board visual inspection," IACR Cryptol. ePrint Arch., vol. 2020, p. 1422, 2020.

[13] R. Solovyev, W. Wang, and T. Gabruseva, "Weighted boxes fusion: Ensembling methods for object detection measures," Image Vis. Comput., vol. 107, p. 104117, 2021.

[14] S. Tang, F. He, X. Huang, and J. Yang, "Online PCB defect detector on a new PCB defect dataset," arXiv preprint arXiv:1902.06197, 2019.

[15] F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee, "Digital twin in industry: State-of-the-art," IEEE Trans. Ind. Informat., vol. 15, no. 4, pp. 2405-2415, 2019.

[16] Z. Tong, Y. Chen, J. Xu, and R. Yu, "Wise-IoU: Bounding box regression loss with dynamic focusing mechanism," arXiv preprint arXiv:2301.10051, 2023.

[17] X. Ye, L. Huang, J. Wang, X. Hu, J. Xu, and C. Zhang, "A lightweight PCB defect detection algorithm based on an improved YOLOv11s," IEEE Access, vol. 14, pp. 56412-56426, 2026, doi: 10.1109/ACCESS.2026.3678967.

[18] Z. Zhang, "Improved Adam optimizer for deep neural network training in industrial visual inspection," IEEE Trans. Ind. Electron., vol. 68, no. 8, pp. 7412-7422, 2021.